



The Norm

Version 4.1

Summary: 본 문서는 42에서 적용되는 프로그래밍 표준(이하 *Norm*)을 설명합니다.
*Norm*은 코드 작성 시 따라야 하는 일련의 규칙들을 정의하며, 기본적으로 공통 과정
내의 모든 C 프로젝트와 *Norm*이 명시된 모든 프로젝트에 적용됩니다.

Contents

I	서문	2
II	왜?	3
III	The Norm	4
III.1	명명 규칙 (Naming)	4
III.2	서식 (Formatting)	5
III.3	함수	7
III.4	사용자 정의 자료형, 구조체, 열거형 및 공용체	8
III.5	헤더 – 즉, include 파일	9
III.6	42 헤더 – 즉, 멋지게 파일 시작하기	10
III.7	매크로 및 전처리기 (Macros and Pre-processors)	11
III.8	금지된 사항! (Forbidden stuff!)	12
III.9	주석 (Comments)	13
III.10	파일 (Files)	14
III.11	Makefile	15

Chapter I

서문

`norminette`는 Python으로 작성된 오픈 소스 코드로, 여러분의 소스 코드가 Norm을 준수하는지를 검사합니다. 이 도구는 Norm의 여러 제약사항들을 검사하지만, 모든 제약사항(예: 주관적인 제약사항 등)을 검사하지는 않습니다. 특정 캠퍼스 내의 지역 규정이 없는 한, 통제 항목 평가 시 `norminette`의 결과가 우선 적용됩니다. 다음 페이지에서는 `norminette`가 검사하지 않는 규칙들에는 (*) 표시가 되어 있으며, 평가자가 코드 리뷰 도중 이를 발견할 경우 (Norm 플래그 사용 시) 프로젝트 실패로 이어질 수 있습니다.

해당 저장소는 <https://github.com/42School/norminette> 에서 확인할 수 있습니다.

풀 리퀘스트, 제안 및 이슈는 언제든지 환영합니다!

Chapter II

왜?

Norm은 다양한 교육적 필요를 충족시키기 위해 신중하게 설계되었습니다. 아래는 Norm의 여러 선택에 대한 가장 중요한 이유들입니다.

- 순차적 실행 (Sequencing): 코딩은 큰 복잡한 작업을 여러 개의 기본 명령어로 분할하는 것을 의미합니다. 이 명령어들은 차례대로, 하나씩 실행됩니다. 소프트웨어 개발을 시작하는 초보자는 모든 개별 명령어와 정확한 실행 순서를 완전히 이해할 수 있는 단순하고 명확한 프로젝트 아키텍처가 필요합니다. 동시에 여러 명령어를 수행하는 듯한 암호 같은 문법이나, 하나의 코드 블록 내에 여러 작업이 혼합된 함수들은 혼란과 오류의 원인이 됩니다.

Norm은 각 코드 조각의 고유한 작업이 명확하게 이해되고 검증될 수 있도록 단순한 코드를 작성할 것을 요구하며, 실행되는 모든 명령어의 순서에 의문의 여지가 없도록 합니다. 이 때문에 함수는 최대 25줄로 제한되며, for, 'do .. while' 또는 삼항 연산자(ternaries)는 금지됩니다.

- 룩 앤 필 (Look and Feel): 동료들과의 피어러닝(peer-learning) 과정 및 피어 평가(peer-evaluations) 시, 다른 사람의 코드를 해독하는 데 시간을 쓰기보다는 바로 코드의 로직에 대해 논의하고 싶을 것입니다.

Norm은 함수와 변수의 명명, 들여쓰기, 중괄호 규칙, 탭과 공백 사용 등 특정 룩 앤 필을 사용하도록 지침을 제공합니다. 이를 통해 익숙하게 보이는 다른 사람의 코드를 빠르게 이해하고, 코드를 읽어야 하는 시간을 줄여 바로 핵심 내용으로 들어갈 수 있습니다. 또한 Norm은 하나의 상표(trademark)로 작용하여, 42 커뮤니티의 일원으로서 취업 시장에서 다른 42 학생이나 동문의 코드임을 인지할 수 있게 합니다.

- 장기적 관점 (Long-term vision): 이해하기 쉬운 코드를 작성하기 위한 노력은 유지보수에 가장 효과적입니다. 올바른 방식으로 코드를 작성하면, 다른 사람이든 본인이든 버그를 수정하거나 새로운 기능을 추가할 때 해당 코드가 무엇을 하는지 파악하는 데 소중한 시간을 낭비하지 않아도 됩니다. 이는 단순히 시간이 많이 소요되어 관리되지 않는 코드 부분들이 발생하는 상황을 방지하며, 시장에서 성공적인 제품을 만드는 데 결정적인 역할을 합니다. 이러한 방식을 빨리 익힐수록 좋습니다.

- 참조 (References): Norm에 포함된 일부 또는 모든 규칙이 임의적이라고 생각할 수도 있으나, 실제로 우리는 “함수는 왜 짧고 하나의 작업만 수행해야 하는지”, “변수의 이름이 왜 의미 있어야 하는지”, “왜 한 줄의 길이가 80 열을 넘으면 안 되는지”, “함수가 왜 많은 매개변수를 받아서는 안 되는지”, “주석은 왜 유용해야 하는지” 등에 대해 충분히 고민하고 연구하였습니다. 이에 대해 구글링해 보시길 강력히 권장합니다.

Chapter III

The Norm

III.1 명명 규칙 (Naming)

- 구조체의 이름은 반드시 s_ 로 시작해야 합니다.
- typedef의 이름은 반드시 t_ 로 시작해야 합니다.
- 공용체(union)의 이름은 반드시 u_ 로 시작해야 합니다.
- 열거형(enum)의 이름은 반드시 e_ 로 시작해야 합니다.
- 전역 변수의 이름은 반드시 g_ 로 시작해야 합니다.
- 변수, 함수, 사용자 정의 타입 등 식별자는 소문자, 숫자, 그리고 언더스코어 '_' (snake_case)만 포함할 수 있으며, 대문자는 허용되지 않습니다.
- 파일 및 디렉토리 이름도 소문자, 숫자, 언더스코어 '_' (snake_case)만 포함해야 합니다.
- 리터럴 문자열 및 문자 내부를 제외하고, 표준 ASCII 테이블에 포함되지 않는 문자는 금지됩니다.
- (*) 모든 식별자(함수, 타입, 변수 등)의 이름은 명시적이어야 하며, 기억하기 쉬운 (mnemonic) 영어로 작성되고, 각 단어는 언더스코어로 구분되어야 합니다. 이 규칙은 매크로, 파일 이름, 디렉토리에도 동일하게 적용됩니다.
- 프로젝트에서 명시적으로 허용하지 않는 한, const나 static으로 지정되지 않은 전역 변수의 사용은 금지되며 Norm 오류로 간주됩니다.
- 파일은 반드시 컴파일되어야 합니다. 컴파일되지 않는 파일은 Norm을 통과할 것으로 기대되지 않습니다.

III.2 서식 (Formatting)

- 각 함수는 함수의 중괄호를 제외하고 최대 25줄 이내여야 합니다.
- 각 줄은 주석을 포함하여 최대 80 컬럼 이내여야 합니다. 주의: 탭 문자는 한 컬럼이 아니라 해당 탭이 표현하는 공백 수만큼 계산됩니다.
- 함수들은 빈 줄로 구분되어야 하며, 함수 사이에는 주석이나 전처리 지시문이 삽입될 수 있지만 최소한 한 줄의 빈 줄이 있어야 합니다.
- 코드는 4문자 길이의 탭 (실제 탭, ASCII CODE 9)으로 들여쓰기를 해야 하며, 이는 4개의 공백과는 다릅니다. 코드 에디터가 올바르게 설정되어 `norminette`에서 요구하는 들여쓰기를 시각적으로 확인할 수 있도록 하세요.
- 중괄호 내의 블록은 들여쓰기를 해야 하며, 구조체, 열거형, 공용체의 선언을 제외하고 중괄호는 각각 독립된 줄에 위치해야 합니다.
- 빈 줄은 공백이나 탭이 포함되어서는 안 되며, 반드시 비어 있어야 합니다.
- 각 줄은 공백이나 탭으로 끝나서는 안 됩니다.
- 연속된 두 개의 빈 줄 또는 공백은 허용되지 않습니다.
- 함수 내의 선언은 반드시 함수의 시작 부분에 있어야 합니다.
- 해당 범위 내에서 모든 변수 이름은 같은 열에 들여쓰기로 정렬되어야 합니다. (참고: 타입은 이미 포함된 블록에 의해 들여쓰기가 적용됩니다.)
- 포인터에 사용되는 별표(*)는 변수 이름에 붙여서 작성해야 합니다.
- 한 줄에는 한 개의 변수 선언만 허용됩니다.
- 전역 변수(허용되는 경우), `static` 변수, 상수를 제외하고, 변수 선언과 초기화는 같은 줄에 있을 수 없습니다.
- 함수 내에서는 변수 선언과 함수의 나머지 코드 사이에 빈 줄을 하나 삽입해야 하며, 그 외의 빈 줄은 허용되지 않습니다.
- 한 줄에는 하나의 명령어나 제어 구조만 있어야 합니다. 예를 들어, 제어 구조 내의 할당, 한 줄에 두 개 이상의 할당, 제어 구조의 끝에 줄바꿈이 없는 경우는 모두 금지됩니다.
- 필요에 따라 명령어나 제어 구조를 여러 줄로 나눌 수 있으며, 이후 줄은 첫 번째 줄보다 들여쓰기를 해야 합니다. 줄을 나눌 때는 자연스러운 공백을 사용하며, 해당되는 경우 연산자는 새 줄의 시작 부분에 위치해야 하고, 이전 줄의 끝에 위치해서는 안 됩니다.
- 줄의 끝이 아닌 모든 쉼표(,)나 세미콜론(;) 뒤에는 공백이 있어야 합니다.
- 각 연산자나 피연산자 사이에는 단 하나의 공백만 있어야 합니다.
- 각 C 키워드 뒤에는 공백이 있어야 합니다. 단, 자료형을 나타내는 키워드(예: `int`, `char`, `float` 등)와 `sizeof`는 예외입니다.

- 제어 구조(if, while 등)는 단일 줄의 단일 명령어를 포함하는 경우를 제외하고는 반드시 중괄호를 사용해야 합니다.

일반적인 예시:

```
int      g_global;
typedef struct  s_struct
{
    char    *my_string;
    int     i;
}          t_struct;
struct     s_other_struct;

int      main(void)
{
    int     i;
    char    c;

    return (i);
}
```

III.3 함수

- 함수는 최대 4개의 명명된 매개변수만 가질 수 있습니다.
- 인자를 받지 않는 함수는 반드시 매개변수 자리에 "void"를 명시하여 프로토타입을 작성해야 합니다.
- 함수 프로토타입 내의 매개변수는 반드시 이름이 지정되어야 합니다.
- 함수 당 5개 이상의 변수 선언은 허용되지 않습니다.
- 각 함수에서 5개를 초과하여 변수를 선언할 수 없습니다.
- 함수의 반환값은 반환할 값이 있을 경우 반드시 괄호로 감싸야 합니다.
- 반환 타입과 함수 이름 사이에는 단 하나의 탭 문자가 있어야 합니다.

```
int my_func(int arg1, char arg2, char *arg3)
{
    return (my_val);
}

int func2(void)
{
    return ;
}
```


III.4 사용자 정의 자료형, 구조체, 열거형 및 공용체

- 구조체를 선언할 때는 다른 C 키워드와 마찬가지로 “struct”와 이름 사이에 공백을 추가해야 하며, 열거형(enum)과 공용체(union)도 동일하게 적용됩니다.
- struct 자료형의 변수를 선언할 때는 변수 이름에 대해 일반적인 들여쓰기 규칙을 적용해야 하며, 열거형과 공용체도 동일합니다.
- 구조체, 열거형, 유니온의 중괄호 내부에서는 일반 블록과 동일한 들여쓰기 규칙이 적용됩니다.
- “typedef” 뒤에도 공백을 추가하고, 새로 정의된 이름에 대해 일반적인 들여쓰기를 적용해야 합니다.
- 해당 스코프 내에서 모든 구조체 이름은 동일한 열에 들여쓰기로 정렬되어야 합니다.
- .c 파일 내에서는 구조체를 선언할 수 없습니다.

III.5 헤더 – 즉, include 파일

- (*) 헤더 파일에 허용되는 요소는 다음과 같습니다. 헤더 포함(시스템 헤더 또는 비 시스템 헤더), 선언, #define, 프로토타입, 매크로
- 모든 #include 문은 파일의 시작 부분에 있어야 합니다.
- 헤더 파일이나 다른 C 파일 내에서 C 파일을 #include 할 수 없습니다.
- 헤더 파일은 중복 포함을 방지하기 위해 보호되어야 합니다. 예를 들어, 파일 이름이 ft_foo.h라면 대응하는 매크로는 FT_FOO_H여야 합니다.
- (*) 사용되지 않는 헤더의 포함은 금지됩니다.
- 헤더 파일 내에서의 헤더 포함은, 필요 시 .c 파일 및 .h 파일 자체에 주석으로 정당화할 수 있습니다.

```
#ifndef FT_HEADER_H
# define FT_HEADER_H
# include <stdlib.h>
# include <stdio.h>
# define FOO "bar"

int          g_variable;
struct      s_struct;

#endif
```

III.6 42 헤더 – 즉, 멋지게 파일 시작하기

- 모든 .c 및 .h 파일은 유용한 정보를 포함한 특별한 형식의 멀티라인 주석으로 구성된 표준 42 헤더로 즉시 시작해야 합니다. 이 표준 헤더는 여러 텍스트 에디터(예: emacs에서는 C-c C-h, vim에서는 :Stdheader 또는 F1 등)에서 클러스터 내의 컴퓨터에 기본적으로 제공됩니다.
- (*) 42 헤더에는 작성자(로그인 및 학생 이메일 (@student.campus)), 작성 날짜, 마지막 업데이트 시의 로그인 및 날짜 등 최신 정보들이 포함되어야 하며, 파일이 디스크에 저장될 때마다 자동으로 업데이트되어야 합니다.



기본 표준 헤더가 여러분의 개인 정보로 자동 구성되지 않을 수 있으므로, 위 규칙을 따르기 위해 수정이 필요할 수 있습니다.

III.7 매크로 및 전처리기 (Macros and Pre-processors)

- (*) 여러분이 생성하는 전처리 상수(또는 #define)는 리터럴 및 상수 값에만 사용되어야 합니다.
- (*) Norm을 우회하거나 코드를 난독화하기 위해 생성된 모든 #define은 금지됩니다.
- (*) 해당 프로젝트 범위 내에서 허용되는 경우에 한해, 표준 라이브러리에서 제공되는 매크로를 사용할 수 있습니다.
- 여러 줄에 걸친 매크로는 금지됩니다.
- 매크로 이름은 모두 대문자로 작성되어야 합니다.
- #if, #ifdef, #ifndef 블록 내의 전처리 지시문은 반드시 들여쓰기를 해야 합니다.
- 전처리 지시문은 전역 범위 외부에서 사용해서는 안 됩니다.

III.8 금지된 사항! (Forbidden stuff!)

- 아래의 구문은 허용되지 않습니다.
 - for
 - do...while
 - switch
 - case
 - goto
- 삼항 연산자(?)의 사용
- 가변 길이 배열(VLAs)
- 변수 선언 시 암시적 타입 사용

```
int main(int argc, char **argv)
{
    int    i;
    char    str[argc]; // 가변 길이 배열

    i = argc > 5 ? 0 : 1 // 삼항 연산자
}
```

III.9 주석 (Comments)

- 주석은 함수 본문 내부에 위치할 수 없으며, 줄의 끝이나 독립된 줄에 있어야 합니다.
- (*) 주석은 영어로 작성되어야 하며, 유용한 정보를 제공해야 합니다.
- 주석은 불필요하게 잡다한 함수(carryall)나 잘못된 함수의 생성을 정당화할 수 없습니다.



잡동사니(carryall) 함수 또는 잘못된 함수는 보통 함수 이름이 f1, f2 등과 같이 명시적이지 않고, 변수 이름이 a, b, c 등으로 되어 있습니다. 오직 Norm을 피하기 위해, 고유한 논리적 목적 없이 작성된 함수도 잘못된 함수로 간주됩니다.

기억해야 할 점은, 함수는 각자 명확하고 단순한 한 가지 작업을 수행하도록 작성하고, 전체 코드는 누가 보아도 이해하기 쉽게 만드는 것이 바람직하다는 것입니다. 한 줄 코드(one-liner)와 같이 코드를 일부러 복잡하고 읽기 어렵게 만드는 기법은 피해야 합니다

III.10 파일 (Files)

- .c 파일 내에서 다른 .c 파일을 #include 할 수 없습니다.
- 하나의 .c 파일에 5개 이상의 함수 정의가 있을 수 없습니다.

III.11 Makefile

Makefile은 norminette에 의해 검사되지 않으며, 평가 지침에 따라 학생이 평가 시 직접 확인해야 합니다. 특별한 지시가 없는 한, 다음 규칙이 Makefile에 적용됩니다.

- $\$(NAME)$, *clean*, *fclean*, *re* and *all* 규칙은 필수이며, *all* 규칙은 기본 규칙으로 단순히 `make` 명령을 입력했을 때 실행되어야 합니다.
- 불필요한 경우에 Makefile이 다시 링크(relink)되면 해당 프로젝트는 기능하지 않는 것으로 간주됩니다.
- 멀티 바이너리 프로젝트의 경우, 위 규칙 외에도 각 바이너리에 대한 규칙(예: $\$(NAME_1)$, $\$(NAME_2)$, ...)을 반드시 포함해야 하며, “all” 규칙은 각 바이너리 규칙을 사용하여 모든 바이너리를 컴파일해야 합니다.
- 소스 코드와 함께 존재하는 비시스템 라이브러리(예: `libft`)의 함수를 호출하는 프로젝트의 경우, 해당 라이브러리를 Makefile에서 자동으로 컴파일해야 합니다.
- 프로젝트 컴파일에 필요한 모든 소스 파일은 Makefile에 명시적으로 이름을 지정해야 하며, 예를 들어 “*.c” 또는 “*.o”와 같은 와일드카드를 사용할 수 없습니다.